

Relational Layer Geometry Improves Compositional Transfer Under a Capability Ceiling

A preregistered synthetic study of geometry, compositional depth, and task-aligned parameter reuse

Alireza Afshan

Independent researcher

19 August 2026

Abstract

Can the geometry of a model's internal representations provide useful supervision to a smaller model, and is such supervision an efficient substitute for the right computational structure? We test these questions on finite-state composition tasks where every example specifies a start entity and a sequence of relations. Our relational objective matches normalized, centered Gram matrices of selected teacher and student encoder-layer states, making the target invariant to orthogonal changes of hidden-state basis. In a prospectively registered affine-permutation confirmation with ten paired student seeds, a generic Transformer trained with teacher logits plus learned-teacher geometry reached 23.31% held-out-pair accuracy, compared with 11.30% for logits alone. The paired effect was 12.01 percentage points (95% CI [10.16, 13.86]). Relational training also retained a larger fraction of above-chance in-distribution performance through increasing depth. However, depth-four held-out accuracy was only 3.51%, and in-distribution depth-four accuracy was only 8.15%; the registered algorithm-transfer criterion failed under a severe student capability ceiling. A task-aligned transition-table executor reached 100% at every depth using 17,672 stored parameters, 61.3 times fewer than the Transformer, but it was given the correct factorization. A fixed bitwise replication missed its teacher positive-control gate and is descriptive only. The result establishes a reproducible benefit from correspondence-preserving learned-teacher layer geometry. It does not yet distinguish teacher-specific internal structure from a simpler target-class similarity objective, identify why deep composition fails, or causally explain the executor's compactness.

1. Introduction

Modern neural networks store large amounts of capability in fixed weights, then create input-dependent activation trajectories at inference time. Those trajectories are attractive compression targets. If two trajectories differ systematically across inputs, it is natural to suspect that the differences encode something useful beyond an arbitrary internal coordinate system. Knowledge distillation provides an operational test: expose a smaller student to selected information from a larger teacher and ask whether held-out behavior improves.

The strongest version of this intuition would be consequential. If internal dynamics specify reusable computation more directly than final outputs do, transferring them might improve capability per parameter. It might also help explain why a large fixed network can produce many context-dependent computations without changing its weights. But a more skeptical possibility is equally plausible: a trajectory is meaningful only together with the teacher's weights, and forcing another architecture to imitate its geometry may transfer superficial correlations rather than the underlying algorithm.

This paper separates three questions that are often blended together:

1. **Coordinate question.** Is any benefit tied to the teacher's particular hidden basis, or can basis-invariant relations among states transfer?
2. **Algorithm question.** Does an activation objective transfer an iterative computation that extrapolates through greater composition depth, or only local regularities?
3. **Efficiency question.** How does richer supervision compare with an architecture that directly represents context-selected parameter reuse?

We study synthetic finite-state transition systems because they make these questions falsifiable on consumer hardware. Each relation is a permutation of a finite entity set. A model receives a start entity and two to four relation tokens, and must predict the result of sequentially applying the corresponding permutations. Some ordered relation pairs are withheld from student training. This design lets us distinguish memorizing familiar local combinations from repeatedly executing a primitive transition rule.

Our target is a normalized centered Gram matrix over selected encoder-layer states in each minibatch. It retains pairwise inner-product geometry while discarding any privileged orthogonal basis. We compare label supervision, teacher logits, pointwise

hidden-state matching, relational matching, shuffled layer representations, fixed random targets, and an untrained teacher. We also compare a generic Transformer with a GRU and a deliberately structured transition-table executor.

The empirical result is useful precisely because it is mixed. Learned-teacher geometry produces a large, seed-consistent improvement over logits in the registered affine task. Correspondence shuffling, random features, and an untrained teacher do not reproduce it. This argues that the particular learned, example-aligned target is useful, but it does not show which information in that target matters. Accuracy falls toward chance as depth increases while in-distribution capability also collapses, so the experiment neither demonstrates a learned iteration rule nor isolates a failure to transfer one. The compact transition executor solves the task exactly because its architecture is handed relation-selected state transitions and repeated application. It is an existence proof about task structure, not a proposal for replacing language models.

Our contributions are:

- a basis-invariant relational activation objective with matched shuffled, random, and untrained-teacher controls;
- a prospectively registered ten-seed confirmation showing improved compositional transfer while rejecting a stronger algorithm-transfer criterion;
- a post hoc capability-conditioned depth analysis that narrows the allowed mechanism claim;
- a distinction among stored parameters, semantically selected parameters, realized computation, and reuse across input steps;
- a 61.3-fold two-model stored-parameter comparison demonstrating the leverage, and the limitations, of encoding the correct finite-state factorization;
- a transparent failed replication gate that prevents a descriptively favorable second task family from being counted as confirmation; and
- an end-to-end reproducible workflow containing preregistrations, frozen configurations, raw per-seed results, analysis code, and a public draft manuscript.

2. Related work

2.1 Knowledge distillation and internal relations

Classical knowledge distillation trains a student against a teacher's softened output distribution rather than hard labels alone ([Hinton, Vinyals, and Dean, 2015](#)). Later work transfers relations among examples or representations. Relational Knowledge Distillation matches distances and angles among learned examples ([Park et al., 2019](#)), while similarity-preserving distillation transfers pairwise activation similarities ([Tung and Mori, 2019](#)). Our objective belongs to this family, but applies the same idea to selected encoder-layer states and tests whether gains persist through task-composition depth. Encoder layers are not assumed to correspond to the task's individual transition steps.

Representation similarity is itself delicate. Centered kernel alignment (CKA) was developed to compare representations in a way that is invariant to orthogonal transformations and isotropic scaling ([Kornblith et al., 2019](#)). We use a normalized centered linear Gram target for the same reason: a coordinate-basis artifact should not be mistaken for transferred structure. This invariance does not make the target fully representation-independent; it only removes a specific family of coordinate choices.

Closer teacher imitation is not guaranteed to improve generalization. Stanton et al. found that common distillation methods can fail to make student predictions more teacher-like and that student fidelity and accuracy need not move together ([Stanton et al., 2021](#)). Menon et al. formalized statistical aspects of distillation and showed that teacher probabilities can provide information unavailable from one-hot labels under suitable conditions ([Menon et al., 2021](#)). These results motivate our matched output-only baselines and our refusal to treat a representation-similarity score as evidence without behavioral transfer.

2.2 Compositional generalization and recurrent computation

Systematic composition remains a difficult neural-network test. SCAN showed that sequence models can perform well on familiar combinations while failing on deliberately novel compositions ([Lake and Baroni, 2018](#)). Mitchell et al. demonstrated that architectural inductive biases materially affect compositional generalization ([Mitchell et al., 2021](#)). Our withheld ordered relation pairs provide a small mechanistic analogue of this problem.

Architectures with repeated computation offer an obvious alternative to merely increasing depth or supervision. The Universal Transformer applies a recurrent transition across positions and time ([Dehghani et al., 2019](#)). Neural Programmer-Interpreters learn programs composed from reusable subprograms ([Reed and de Freitas, 2016](#)). Conditional-computation systems such as the Switch Transformer activate only selected parameter blocks for each input ([Fedus, Zoph, and Shazeer, 2022](#)). Our transition-table executor

is much simpler: the relation token chooses one transition matrix, and the same update rule is applied at each step. Its value here is diagnostic clarity.

2.3 Scope relative to model compression

This is not a study of large language models, pruning, quantization, natural-language reasoning, or measured energy. Parameter count is an incomplete efficiency measure: memory layout, arithmetic intensity, conditional execution, hardware kernels, and data movement all matter. We therefore report wall time and parameters separately and make no joule claim. The experiment asks a prior mechanistic question: whether a chosen form of internal geometry transfers more useful compositional information than outputs, and whether supervision or factorization is the larger lever on a controlled task.

3. Research questions and registered claims

The work began with E001-P0, an exploratory pilot. E001-P1 then prospectively registered a larger-student capability gate. When every student failed that gate, we stopped the planned confirmation rather than evaluating the held-out confirmatory pairs. This invariant failure motivated E002 and its new preregistration.

E002 registered two principal claims.

Relational-transfer claim. In the affine main cell, relational supervision is supported only if paired 95% Student-t confidence intervals for relational minus logits and relational minus shuffled accuracy exclude zero on the positive side.

Strong algorithm-transfer claim. In addition to the previous rule, the relational advantage must be positive at depths three and four, and mean depth-four relational accuracy must exceed chance by at least ten percentage points.

Structured-efficiency claim. The transition executor must exceed 95% at depths three and four in both the affine and bitwise families, beat the generic baselines in absolute held-out accuracy, and use at least twenty times fewer stored parameters than the Transformer.

The preregistration also imposed a positive-control gate: the teacher must exceed 95% overall and at every confirmatory depth in each task-family cell. A failed gate invalidates that entire cell. This rule matters for the bitwise results below.

4. Methods

4.1 Transition-composition tasks

Let the entity set contain E discrete states and let each relation r denote a permutation T_r of those states. An input contains a start state s_0 and a sequence $(r_1, \dots, r_d)$. The target is

$$s_t = T_{r_t}(s_{t-1}), \text{ for } t = 1, \dots, d.$$

The affine family contains 47 entities and eight relations generated from invertible affine maps. The fixed replication contains 64 entities and eight relations generated by bit rotations followed by XOR masks. Student training uses depths one through four but excludes designated development and confirmatory ordered relation pairs. Confirmatory evaluation contains the frozen confirmatory pairs and excludes development pairs. Random-guess accuracy is $1/47 = 2.13\%$ for affine and $1/64 = 1.56\%$ for bitwise.

The confirmatory datasets contain 20,000 student-training examples, 80,000 teacher-training examples, 10,000 in-distribution examples, and 10,000 confirmatory-pair examples per family. The affine task uses data seed 5150 and student seeds 5101-5110. The bitwise family uses data seed 27182 and student seeds 6101-6105.

4.2 Teacher and generic students

The teacher is a six-layer Transformer encoder with approximately 4.77 million parameters. The generic student is a two-layer, width-256 Transformer encoder with approximately 1.08-1.09 million parameters depending on vocabulary size. A width-32 GRU supplies a small generic recurrent baseline. All models predict the terminal state from the same tokenized input. Conditions share examples, evaluation sets, and paired student seeds.

4.3 Relational layer-geometry objective

For one teacher or student layer, collect a batch-state matrix H whose rows are example representations. With batch size B , define the centering matrix

$$C = I - (1/B) \mathbf{1}\mathbf{1}^T.$$

The normalized centered Gram representation is

$$G(H) = C H H^T C / (||C H H^T C||_F + \epsilon).$$

The student contributes its [CLS] representation after each of two encoder layers. The six-layer teacher contributes layers zero and five, selected by an evenly spaced layer-matching rule fixed in the implementation. These are network-depth indices, not intermediate states of the task's data-generating transition sequence.

The relational loss sums squared Frobenius distances between matched teacher and student Gram matrices. The Gram matrix is unchanged by replacing H with HQ for any orthogonal Q , and normalization removes isotropic scale. It is not invariant to all invertible transformations, nor does equality of Gram matrices establish functional equivalence. Because the final selected teacher layer is also the classifier input, its Gram may encode terminal-answer similarity; the original experiment did not include a target-label Gram control.

The main relational condition combines labels, teacher logits, and this geometry with the development-selected relational weight 3.0. Controls use the same objective and weight but replace the teacher geometry with per-minibatch shuffled teacher examples, fixed random features of matched shape, or geometry from an untrained teacher. The output-only condition receives teacher logits without geometry. Per-minibatch shuffling is reproducible under each run seed but supplies a changing wrong correspondence, so relational-minus-logits is cleaner than treating shuffled performance as a neutral zero-effect baseline.

4.4 Context-selected transition executor

The transition-table model learns one $E \times E$ logit matrix A_r per relation. Its state is a distribution p_t over entities. At each input step, the relation token selects one table and applies

$$p_t = p_{(t-1)} \text{softmax}(A_{(r_t)}).$$

The execution rule is reused at every depth. The affine model stores 17,672 parameters (eight times 47 squared), of which one 2,209-logit table is semantically selected per relation step. The bitwise model stores 32,768 parameters and semantically selects 4,096 per step. This is a factorization count, not a measured compute count: the current reference implementation computes the softmax of all relation tables before indexing the selected table. By comparison, every parameter in the dense student Transformer participates in an ordinary forward pass, so its stored count is about 1.08 million.

This architecture nearly states the finite-state factorization of the problem. Its success can show that the task admits a much more parameter-efficient representation; it cannot show that such a decomposition can be discovered in unstructured domains. Its storage grows as $O(R \cdot E^2)$.

4.5 Development, freezing, and stopping

Development selected five epochs for the transition table and an auxiliary weight of 3.0 for relational objectives. The five-epoch no-depth-one ablation failed, but a clearly labeled post-observation diagnostic reached 100% at ten epochs without primitive examples. That diagnostic did not alter the confirmatory budget.

The exact confirmation configuration files were frozen and hashed before execution. The affine SHA-256 was f683ea11d6c5d854dcc94c19cd13d03765e4ceaa3c8579265623484dfe42b201; the bitwise SHA-256 was 5bb451f0db20fe6f2a686d17e2afe6fa905284a67cdc705f9c34f02597a225ae. The registered stopping rule required us to end after development selection, affine confirmation, fixed bitwise replication, the no-depth-one ablation, and prespecified statistics, without rescue architectures or retuning.

4.6 Statistical analysis and efficiency reporting

For registered relational contrasts, we subtract paired-seed accuracies and form two-sided 95% Student-t confidence intervals over the seed-level differences. We report sample standard deviations for condition summaries. No multiplicity adjustment was registered; the decision is based on explicitly named contrasts rather than a post hoc search.

We report stored parameters, semantically selected parameters per step, wall-clock training time, and absolute accuracy separately. Accuracy per parameter is discussed only alongside a capability floor. Selected parameters do not imply that sparse compute is realized by the implementation. GPU peak allocation is not used as an energy proxy, and the host exposed no accepted joule telemetry. Cached float32 relational targets are also not free: for 20,000 examples, two student layers, and width 256, the layer-representation tensor is approximately 40.96 MB, compared with 3.76 MB of affine teacher logits and roughly 1.28 MB for the token/label/depth tensors under the implementation's fixed-width integer representation.

5. Preliminary experiments and the failed E001-P1 gate

E001-P0 used three exploratory seeds. Relational distillation reached 8.77% held-out ordered-pair accuracy, versus 4.23% for logits, 5.33% for pointwise hidden matching, 3.88% for shuffled geometry, 3.21% for labels, and 2.13% chance. The relational benefit was concentrated at depth two; depth-four accuracy was near chance. These observations motivated a capability-gated confirmation rather than a direct claim.

E001-P1 prospectively required a generic student to reach 80% in-distribution accuracy before testing fresh confirmatory pairs. The teacher reached 99.60% in distribution, 99.57% on development pairs, and 99.21% at development depth four. None of five student budgets or architectures passed. Two-layer width-256 students trained for 24, 48, and 72 epochs plateaued between 43.22% and 43.84% in-distribution accuracy. Four-layer width-128 and width-256 students reached 43.68% and 43.62%, respectively. Development-pair accuracy remained between 3.27% and 5.11%.

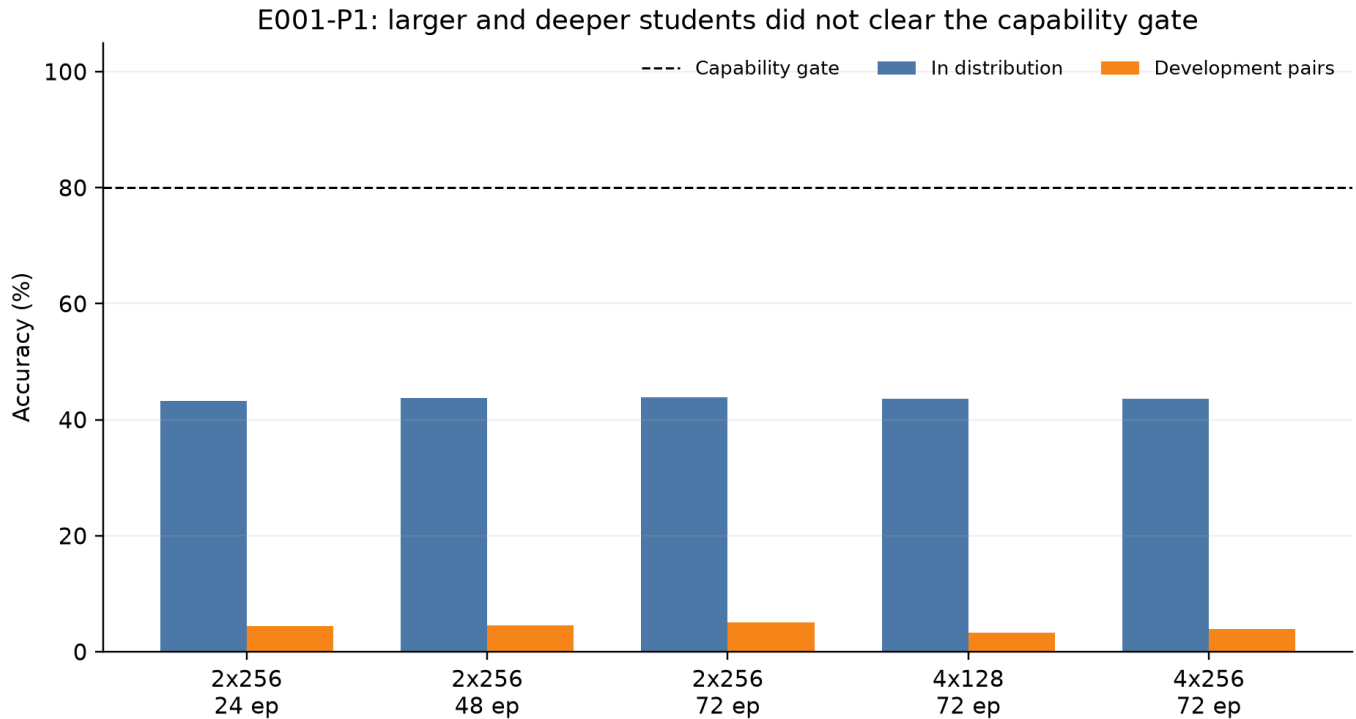


Figure 1. E001-P1 capability gate. Increasing epochs, depth, or width did not move generic students toward the registered 80% gate.

This was a design failure, not negative confirmation of the trajectory hypothesis. The students had not learned the underlying task well enough for an OOD comparison to discriminate among supervision methods. The similar plateau across parameter counts suggested an inductive-bias problem and motivated the executor comparison.

6. Affine confirmation

6.1 Positive control and absolute results

The affine teacher passed its gate: confirmatory accuracy was 99.25% overall, with 100.00%, 99.88%, and 98.45% at depths two, three, and four. Table 1 contains the registered student outcomes.

Table 1. Affine confirmatory-pair accuracy across ten seeds. Values are mean +/- sample SD; depth columns are means.

Condition	Stored params	Overall	Depth 2	Depth 3	Depth 4	Train time
Transition table, labels	17,672	100.00 +/- 0.00	100.00	100.00	100.00	1.75 s
GRU, labels	9,647	9.44 +/- 0.88	22.57	8.54	4.42	33.43 s
Transformer, labels	1,082,927	9.99 +/- 2.64	36.56	4.83	2.34	29.79 s
Transformer, logits	1,082,927	11.30 +/- 1.82	41.69	5.61	2.39	34.66 s
Transformer, relational	1,082,927	23.31 +/- 2.02	77.41	18.39	3.51	42.45 s
Transformer, shuffled	1,082,927	6.65 +/- 1.45	22.10	3.59	2.25	42.91 s
Transformer, random	1,082,927	3.93 +/- 0.38	9.53	2.95	2.24	42.84 s
Transformer, untrained	1,082,927	8.34 +/- 1.59	28.49	4.55	2.45	42.22 s

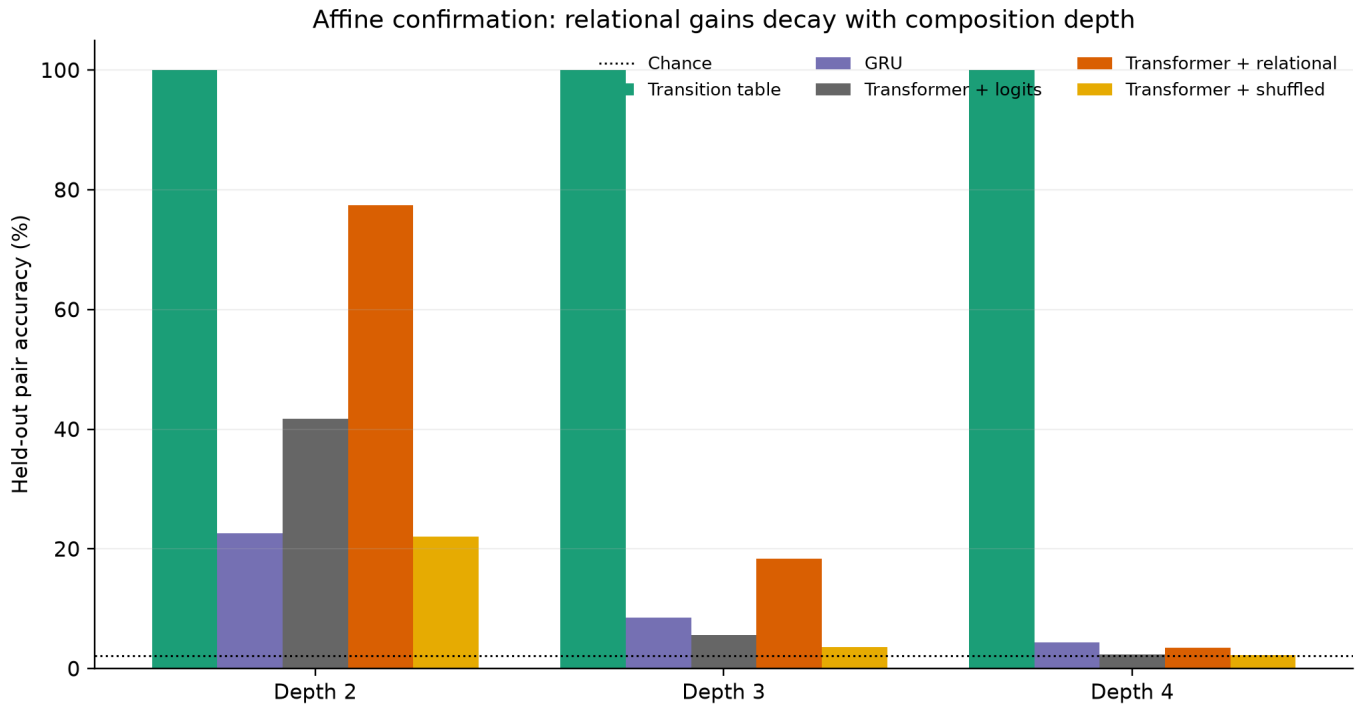


Figure 2. Affine accuracy by composition depth. Relational geometry greatly improves depth-two and depth-three transfer, but the advantage decays toward chance at depth four.

6.2 Registered relational contrasts

Relational supervision beat logits for every paired seed. The overall paired difference was 12.01 percentage points, 95% CI [10.16, 13.86]. It also beat shuffled geometry by 16.66 points [15.04, 18.28]. At depth three the corresponding effects were 12.78 [11.21, 14.35] and 14.80 [13.16, 16.45]. At depth four the intervals remained positive but small: 1.13 [0.70, 1.55] versus logits and 1.27 [0.89, 1.65] versus shuffled.

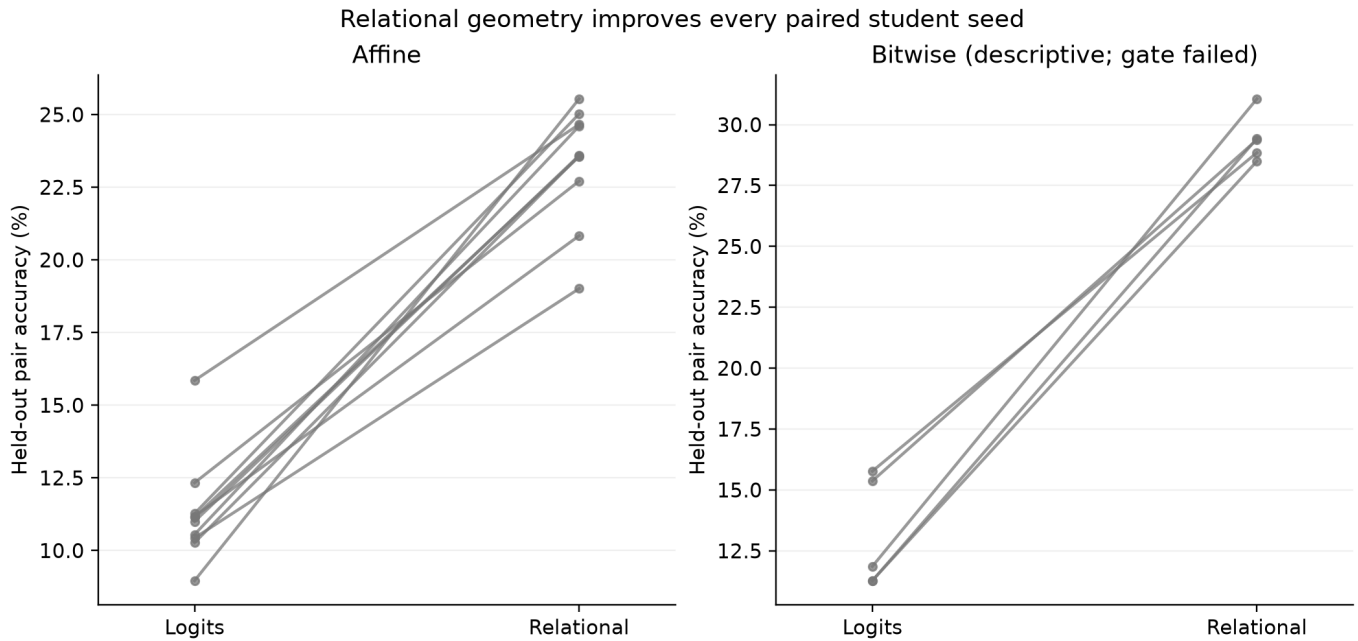


Figure 3. Seed-paired output-only and relational results. Every line rises in the valid affine confirmation. The bitwise panel is descriptive because its teacher gate failed.

The preregistered relational-transfer claim is therefore supported in the affine cell. Three controls sharpen, but do not settle, the interpretation. Shuffling teacher representations breaks the minibatch relation between each input and its target geometry and removes the gain. Fixed random features do not recover it. Geometry from an untrained teacher also performs far below learned-teacher geometry. Together with orthogonal-basis invariance, these results show that the correspondence-preserving

learned-teacher target matters. They do not distinguish distinctive internal computation from terminal-answer similarity already present in the learned representations. A target-label Gram is the smallest missing control.

The stronger algorithm-transfer claim is not supported. Although the paired depth-four interval is positive, relational accuracy is only 3.51%. The registered absolute threshold was chance plus ten percentage points, or 12.13%; the result misses it by 8.62 points. This establishes that the trained student does not execute reliable depth-four behavior. It does not establish that an otherwise learned local algorithm failed specifically during out-of-distribution reuse.

6.3 Capability-conditioned depth analysis

The original endpoints did not report the generic students' in-distribution depth curve. Table 2 adds that post hoc diagnostic from the frozen result artifact.

Table 2. Affine generic-student in-distribution accuracy and descriptive normalized transfer. rho is (heldout - chance) / (in-distribution - chance).

Condition	ID overall	ID depth 2	ID depth 3	ID depth 4	rho 2	rho 3	rho 4
Labels	60.84%	89.86%	27.19%	5.17%	0.392	0.108	0.070
Logits	61.77%	91.09%	29.49%	5.54%	0.445	0.127	0.076
Relational	68.55%	97.00%	49.87%	8.15%	0.793	0.341	0.230

Relational training improves held-out behavior even after normalizing descriptively by the available above-chance in-distribution performance. This strengthens the behavioral-transfer result. At the same time, 8.15% in-distribution depth-four accuracy means that depth four lies under a severe student capability ceiling. The registered failure remains valid, but its mechanism is inconclusive: insufficient capacity, optimization, architecture, and insufficient target information remain live explanations. The rho calculation was prompted after seeing the results, has unstable denominators near chance, and is not a new confirmatory endpoint.

7. Task-aligned parameter reuse

The transition table reaches 100% on every affine confirmatory depth with 17,672 parameters and about 1.75 seconds of training. It uses 61.28 times fewer stored parameters than the relational Transformer. One 2,209-parameter table is semantically selected at each relation step, but the reference forward pass computes softmax probabilities for all eight tables before indexing; no 490-fold realized-compute claim follows. Its absolute-accuracy-per-stored-parameter ratio is about 263 times that of the relational Transformer. Wall time is roughly 24 times lower in this implementation.

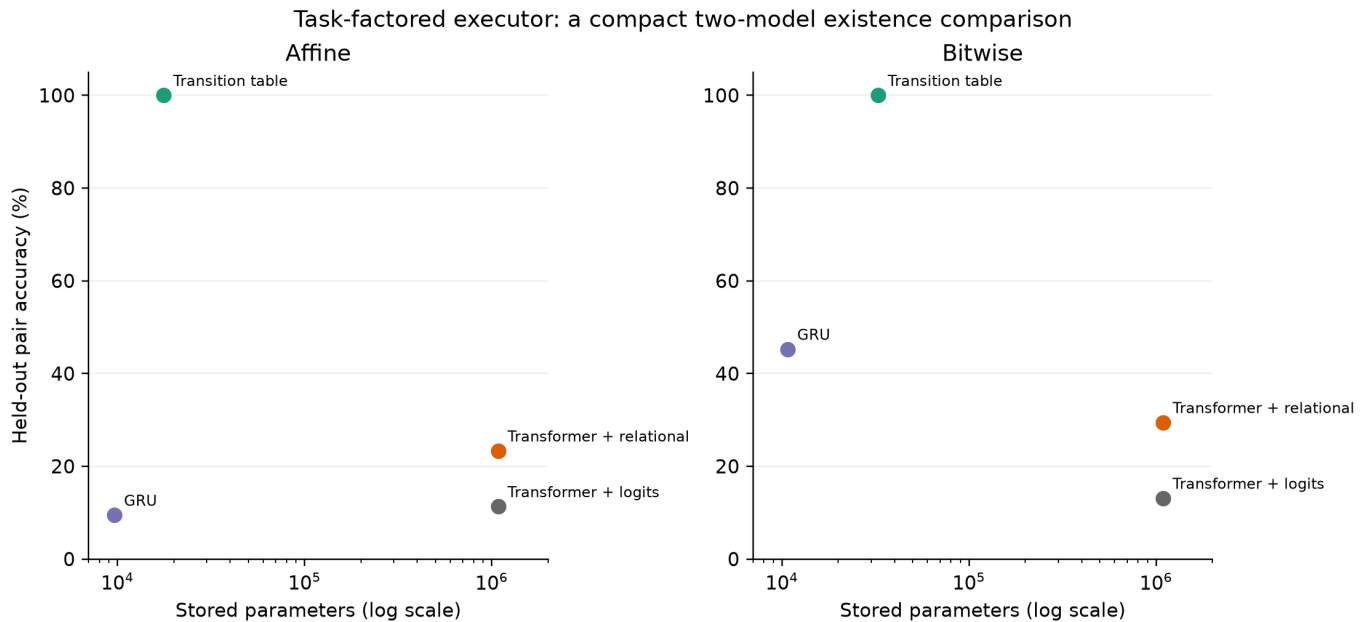


Figure 4. Held-out accuracy versus stored parameters for the measured models. The task-factored executor is a compact existence comparison, not a swept efficiency frontier. Bitwise values are descriptive only.

Those ratios should not be read as a universal compression result. The executor was given the correct state space, relation inventory, and recurrence structure. Its tables store all possible state transitions, including many entries that the affine algebra could encode even more compactly. Conversely, the Transformer must infer the factorization from examples and use an encoder architecture that

was not designed for iterative execution. The comparison answers an existence question: on this task, the needed behavior is representable with an order of magnitude fewer parameters once the right conditional computation is explicit. Because this is a comparison of selected architectures rather than a parameter/data/compute sweep, it does not estimate a general Pareto frontier or show that context selection itself caused the gap.

The no-depth-one diagnostic adds nuance. At the registered five epochs, omitting single-relation examples yielded only 6.32%-11.60% held-out accuracy across three development seeds. Without changing the confirmatory configuration, an exploratory extension showed 100% at ten epochs and above. Primitive-transition labels were therefore not logically necessary, but removing them doubled the observed optimization budget. The task factorization, not merely direct single-step supervision, explains the executor's eventual exact composition.

The generic GRU is an important counterexample to a simplistic recurrence story. It has fewer parameters than the transition table but reaches only 9.44% on affine. Reusing a hidden update is not sufficient; the form of the reusable update must align with the task or be learnable under the available data and optimization.

8. Fixed bitwise replication and failed positive control

We applied the frozen hyperparameters to the 64-state bitwise family with five new seeds and no retuning. The teacher achieved 94.46% overall, 98.90% at depth two, 98.87% at depth three, and 89.21% at depth four. Because the registered rule required at least 95% at every depth, the positive control failed. The entire bitwise distillation cell is formally invalid. We did not retrain the teacher, alter the threshold, or add a rescue run.

For transparency, Table 3 reports the resulting measurements descriptively. They must not be treated as a confirmatory replication of relational transfer or structured efficiency.

Table 3. Bitwise measurements across five seeds. The teacher gate failed; all values are descriptive, not confirmatory.

Condition	Stored params	Overall	Depth 2	Depth 3	Depth 4	Train time
Transition table, labels	32,768	100.00 +/- 0.00	100.00	100.00	100.00	2.29 s
GRU, labels	10,752	45.23 +/- 3.29	64.63	48.40	34.39	36.88 s
Transformer, labels	1,091,648	10.08 +/- 2.06	29.85	7.89	3.10	39.50 s
Transformer, logits	1,091,648	13.11 +/- 2.26	37.46	11.43	3.75	41.45 s
Transformer, relational	1,091,648	29.44 +/- 0.98	69.86	32.48	9.52	43.69 s
Transformer, shuffled	1,091,648	8.38 +/- 0.99	24.75	6.54	2.62	43.56 s
Transformer, random	1,091,648	4.03 +/- 1.08	7.80	4.26	2.21	43.84 s
Transformer, untrained	1,091,648	16.96 +/- 3.14	46.02	15.64	5.27	43.28 s

The descriptive relational-minus-logits difference was 16.33 points, 95% CI [13.03, 19.64], and relational-minus-shuffled was 21.06 [20.17, 21.96]. Depth-four relational accuracy was 9.52%, above 1.56% chance but below the unchanged 11.56% strong-algorithm threshold. The GRU's 45.23% is also notable: unlike in affine, generic recurrence aligns partially with this family. These observations are useful for designing a future independently gated replication, but they cannot repair the failed positive control.

One might argue that the labels-only transition table does not mechanically depend on the teacher and should be exempt from a teacher gate. That narrower reinterpretation was not preregistered. The protocol declared the family cell invalid, so we preserve that boundary here. The 100% bitwise table result is an observation, not registered cross-family support.

9. Discussion

9.1 What is present in the trajectories?

The affine result rejects an overly strong null hypothesis: a student's behavior cannot benefit from relations among a teacher's layer representations unless it also receives the teacher's weights. A basis-invariant Gram target improves held-out behavior, and the effect disappears when the example-to-geometry assignment is shuffled or when learned-teacher geometry is replaced with random or untrained features. The safest interpretation is that correspondence-preserving relations among learned teacher representations provide a useful training signal.

That statement is weaker than saying the target contains teacher-specific internal computation. The final selected teacher layer directly feeds the classifier, so its example geometry may largely encode terminal-answer classes. Shuffled, random, and untrained controls do not test that rival. The effect could reflect supervised metric learning expressed through a learned teacher rather than additional algorithmic content in its activations. E003 prospectively registers the missing target-label Gram comparison.

The depth curve also admits fewer mechanistic claims than the registered behavioral decision. Relational supervision improves the descriptive normalized-transfer ratio at all three depths, but the student's in-distribution capability collapses with depth. The current evidence cannot distinguish missing target information from insufficient capacity, representational impossibility, or a severe optimization/inductive-bias mismatch. It shows that the frozen relational objective did not produce reliable depth-general behavior; it does not show that a learned algorithm was present locally and then became non-portable.

9.2 Context-aware parameter definitions

The experiment suggests a useful vocabulary for the project's efficiency question.

- **Stored parameters** measure persistent learned scalars.
- **Semantically selected parameters** are the subset identified by the model's factorization as relevant to a computation step.
- **Realized computation** is what the implementation and hardware actually evaluate, including dense work performed before or around selection.
- **Reused parameters** are applied repeatedly as the input demands more computation.
- **Effective capability** must still be measured at an absolute behavioral floor.

Two models with the same stored count can have very different computation; two models with different stored counts can implement the same algorithm through different factorizations. The transition table uses more stored parameters than the small GRU, yet its relation token semantically selects an interpretable operator and its repeated update exactly matches the data-generating process. The current code still normalizes every operator before selection, illustrating why semantic selection and realized sparse compute must not be conflated. Thus "capability per parameter" is incomplete unless the denominator, implementation, and selection/reuse mechanism are stated.

This is conceptually adjacent to conditional computation and modular networks, but the experiment does not establish that sparse expert routing or dynamic weights improve language-model efficiency. It generates a narrower hypothesis: compression gains may come less from retaining every detail of a large model's activations and more from discovering a compact set of reusable, context-selected operators.

9.3 Implications for compression

No general lossless compression claim follows. The table's 61.3-fold stored-parameter advantage comes from known finite-state structure, two selected model designs, and a tiny domain. In realistic tasks, the correct states may be latent, relation boundaries ambiguous, transition operators continuous, and error accumulation costly. A table also scales quadratically with state count. The result is best treated as a target for representation discovery: can a learner infer a small operator library and execution rule without being handed the factorization?

Trajectory supervision may still help that discovery. Our data suggest it carries relational hints even when it fails to transfer the full procedure. A future architecture could use geometry to learn state abstractions or routing assignments while a recurrent executor supplies the missing iteration. That is a new hypothesis, not a result of this paper.

9.4 Failed gates as evidence

Two failures materially shaped the conclusion. E001-P1 stopped because its students missed the capability gate; this prevented a misleading OOD comparison among incapable models. The bitwise cell stopped at interpretation because its teacher missed the positive-control gate; this prevented descriptively favorable numbers from being called replication. These rules reduced the number of positive claims but increased their auditability.

10. Limitations

First, the tasks are synthetic and finite. They do not establish effects in language models, continuous control, perception, continual learning, or natural data. Second, the transition executor receives the correct factorization and known entity identities; representation discovery is excluded. Its semantic table selection is not realized as sparse compute by the current code. Third, the generic baselines were not exhaustively tuned, and a different recurrent or iterative architecture might close the gap. The registered stopping rule intentionally forbids post-result rescue searches.

Fourth, the affine relational confirmation uses ten paired seeds, but only one task-generation seed and one family passed all confirmatory gates. Generalization across task families is unresolved. Fifth, the bitwise teacher failure makes even favorable student contrasts nonconfirmatory. Sixth, Student-t intervals at $n=10$ summarize seed variation but do not include uncertainty across dataset generation, architecture choice, or researcher decisions.

Seventh, the relational target is only invariant to orthogonal basis changes and isotropic scaling. Other invertible reparameterizations can alter it. The teacher was trained without the student's pair exclusions and is therefore a deliberately privileged supervisor. Eighth, the final selected teacher layer may expose answer-class geometry, and no target-label Gram was included. The shuffled and random controls are destructive alternatives rather than neutral controls. Ninth, matched example counts and model sizes do not imply matched information or compute: layer-representation tensors are larger than logits, relational objectives add training time, and E001's registered data-rich and wall-time-matched label baselines were never run after its capability gate failed. Tenth, wall time on one RTX 4060 is an implementation-specific systems observation, not an energy measurement. Finally, no causal intervention showed that a particular geometric relation mediates behavior; the controls establish predictive utility of the supervision signal, not a mechanistic identity.

11. Reproducibility, ethics, and provenance

All experiment code, exact configurations, preregistrations, selection records, raw per-seed JSON, generated statistics, and manuscript sources are included in the repository. The main runs used Python 3.12, PyTorch 2.13.0 with CUDA 13.0, and an NVIDIA RTX 4060 on Windows. Deterministic tests cover task generation, held-out pair separation, Gram invariance, control construction, model parameter accounting, and executor behavior. Configuration hashes and pre-run git states are recorded in the raw results.

The work uses synthetic data and presents no human-subject or privacy risk. It does not test consciousness, theory of mind, deception, autonomous agency, or deployment behavior. The likely ethical risk is epistemic: extrapolating a clean finite-state demonstration into unsupported claims about brains or large language models. We mitigate that risk by stating failed gates, absolute performance, and scope boundaries prominently.

OpenAI Codex assisted with repository inspection, implementation, experiment execution, statistical analysis, figure generation, and manuscript drafting under the human research direction of Alireza Afshan. The author is responsible for reviewing the design, claims, and final text. No language model was used as an experimental subject or a source of empirical labels.

12. Conclusion

Normalized learned-teacher layer geometry more than doubles the generic Transformer's held-out-pair accuracy relative to output distillation in the affine composition task. The effect is reproducible across paired seeds, survives an orthogonal-basis-invariant formulation, and is not reproduced by shuffled, random, or untrained targets. It also improves descriptive capability-conditioned transfer through depth. The current evidence nevertheless does not identify the useful information as teacher-specific: terminal-label similarity is an untested rival. The registered strong algorithm criterion fails under a severe depth-four capability ceiling, leaving the cause of deep-composition failure inconclusive.

A task-factored transition executor solves the same task exactly with 61.3 times fewer stored parameters. That result is deliberately unfair in an informative way: the executor knows the correct factorization. It proves that a compact exact representation exists on this task. It does not establish a general efficiency frontier, a causal benefit from context selection, or realized sparse compute.

The immediate experiment is the cheaper specificity test: compare learned-teacher geometry with target-label geometry under the frozen affine setup. Any language-model extension should then include both targets, output distillation, and an ordinary extra-compute or extra-data baseline. A separate architecture study can ask whether a learner can discover a compact operator library and recurrent execution rule rather than receiving one.

References

- Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J., and Kaiser, L. (2019). [Universal Transformers](#). ICLR.
- Fedus, W., Zoph, B., and Shazeer, N. (2022). [Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity](#). JMLR, 23.
- Hinton, G., Vinyals, O., and Dean, J. (2015). [Distilling the knowledge in a neural network](#). NeurIPS Deep Learning Workshop.
- Kornblith, S., Norouzi, M., Lee, H., and Hinton, G. (2019). [Similarity of neural network representations revisited](#). ICML.
- Lake, B. M., and Baroni, M. (2018). [Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks](#). ICML.
- Menon, A. K., Rawat, A. S., Reddi, S. J., Kim, S., and Kumar, S. (2021). [A statistical perspective on distillation](#). ICML.
- Mitchell, M., Bowers, J., Dolsak, N., et al. (2021). [Strong inductive biases provably prevent compositional generalization](#). ICML Workshop on Overparameterization.
- Park, W., Kim, D., Lu, Y., and Cho, M. (2019). [Relational knowledge distillation](#). CVPR.

Reed, S., and de Freitas, N. (2016). [Neural Programmer-Interpreters](#). ICLR.

Stanton, S., Izmailov, P., Kirichenko, P., Alemi, A. A., and Wilson, A. G. (2021). [Does knowledge distillation really work?](#). NeurIPS.

Tung, F., and Mori, G. (2019). [Similarity-preserving knowledge distillation](#). ICCV.

Appendix A. Registered decision ledger

Decision	Rule	Outcome
E001-P1 capability	At least 80% student ID accuracy before confirmation	Failed; confirmatory pairs not evaluated
Affine teacher	At least 95% overall and at every depth	Passed
Affine relational transfer	Positive paired CIs versus logits and shuffled	Supported
Affine strong algorithm transfer	Positive depth CIs and depth-four at least chance + 10 points	Not supported
Affine structured executor	Above 95% at depths 3-4, beats baselines, at least 20x fewer parameters	Affine portion passed
Bitwise teacher	At least 95% overall and at every depth	Failed at depth four; cell invalid
Cross-family structured efficiency	Registered criteria in both families	Inconclusive because bitwise cell invalid

Appendix B. Exact run inventory

- E001-P0: three exploratory seeds; six student conditions.
- E001-P1 development: five registered architecture/budget trials, three seeds each; stopped at capability gate.
- E002 affine confirmation: eight conditions, ten paired student seeds, one passing teacher positive control.
- E002 bitwise fixed replication: eight conditions, five paired student seeds, failed teacher positive control.
- E002 development: three transition-table budget seeds, three no-depth-one seeds, four relational-weight settings over three seeds, and explicitly labeled post-observation diagnostics.

All figures and tables are regenerated by `paper/analyze.py` directly from committed raw JSON. The source-of-truth result files are listed in `paper/README.md`.